



Algorithms and Data Structures

Sujeong Kim



UNIVERSITY OF
MARYLAND

Order Statistic

Slide Credit: These slides are adapted from materials by Prof. Manocha, with modifications and additions by Sujeong Kim.

Order Statistic

- A value from a sample of random variables that gives insight into the distribution of the sample (e.g., the value at a particular position when the sample is sorted in ascending order)
- Types of Order Statistics:
 - Minimum: the first order statistic
 - Maximum: the n -th order statistic
 - Median: the middle value when the sample is sorted
 - For an odd number of elements: order statistics at position $\text{floor}((n+1)/2)$
 - For an even number of elements: average of the two middle values, which are the order statistics at position $\text{floor}((n+1)/2)$ and $\text{ceil}((n+1)/2)$
 - i -th smallest element: the i -th order statistic

Finding Minimum in Unsorted Array

1. $\text{min} \leftarrow A[1]$
2. for $i \leftarrow 2$ to $\text{length}[A]$
3. do if $\text{min} > A[i]$
4. then $\text{min} \leftarrow A[i]$
5. return min

Algorithm Analysis – Time Complexity

- $T(n) = \Theta(n)$ for Minimum(A) or Maximum(A)
- Line 4 is executed $\Theta(\lg n)$
 1. $\text{min} \leftarrow A[1]$
 2. for $i \leftarrow 2$ to $\text{length}[A]$
 3. do if $\text{min} > A[i]$
 4. then $\text{min} \leftarrow A[i]$
 5. return min

Algorithm Analysis – Line 4

- Let s_i the indicator variable:
 - $s_i = 1$ if $A[i]$ is the minimum among all elements $A[1], A[2], \dots, A[i]$ (note: not among all n elements)
 - $s_i = 0$ otherwise
- The probability of $s_i = 1$ is $\frac{1}{i}$
- Expected value of s_i
 - If you repeated this random process many, many times, what fraction of the time would s_i be 1?

$$E[s_i] = P(s_i = 1) = \frac{1}{i}$$

Algorithm Analysis – Line 4

- The sum $E[s_1 + s_2 + \dots + s_n]$ calculates the expected total number of times the condition is met (for line 4) across all indices

$$\begin{aligned} E[s] &= E[s_1 + s_2 + \dots + s_n] \\ &= \frac{1}{1} + \frac{1}{2} + \dots + \frac{1}{n-1} + \frac{1}{n} = \sum_{i=1}^n \frac{1}{i} \\ &= \ln n + O(1) = \Theta(\lg n) \end{aligned}$$

Algorithm Analysis – Line 3

- Number of comparisons
 - Best case? $n - 1$
 - Worst case? $n - 1$
 - Average case? $n - 1$
- Can we find the maximum using fewer than $n-1$ comparisons?

Finding Both Max and Min

- Naïve solution:
 - Find max with $n-1$ comparisons
 - Find min with $n-1$ comparisons
 - Total $2n-2$ comparisons
- Can we find the minimum and maximum using fewer than $2(n-1)$ comparisons?

Finding Both Max and Min - Pairwise Approach (1)

- Group elements into pairs
- Compare within the pair: $\lceil n/2 \rceil$ pairs
- Compare the smaller element of the pair with the current minimum
- Compare the larger element of the pair with the current maximum
- Example: [4, 7, 1, 9, 2, 5]

4	7
---	---

min=4
max=7

1	9
---	---

min>1?
max<9?

2	6
---	---

min>2?
max<6?

Number of comparisons

$$\frac{n}{2} + (\frac{n}{2} - 1) + (\frac{n}{2} - 1) = \frac{3n}{2} - 2$$

Finding Both Max and Min - Pairwise Approach (2)

- If n is odd,
 - We find max and min of first $n-1$ items
 - Compare the last element separately with the current min and max
- Example: [4, 7, 1, 9, 2, 5]



min=4
max=7



min>1?
max<9?



min>2?
max<6?



min>10?
max>10?

Number of comparisons

$$\frac{n-1}{2} + \left(\frac{n-1}{2} - 1\right) + \left(\frac{n-1}{2} - 1\right) + 2 = \frac{3(n-1)}{2}$$

Finding Both Max and Min

- Number of Comparisons
 - If n is odd: $\frac{3(n-1)}{2}$
 - If n is even: $\frac{3n}{2} - 2$ comparisons
 - $3\lceil n/2 \rceil$ comparisons (better than finding max and min separately)

Selection Problem

Selection Problem

- Finding the i -th smallest element in an unsorted list
- Input: a set A of n distinct numbers and a number i , with $1 \leq i \leq n$
- Output: the element $x \in A$ that is larger than exactly $i - 1$ other elements of A

Intuitive Approach

- Sort the entire array (e.g., Merge sort) then jump to the i th element

$$O(n \lg n) + O(1) = O(n \lg n)$$

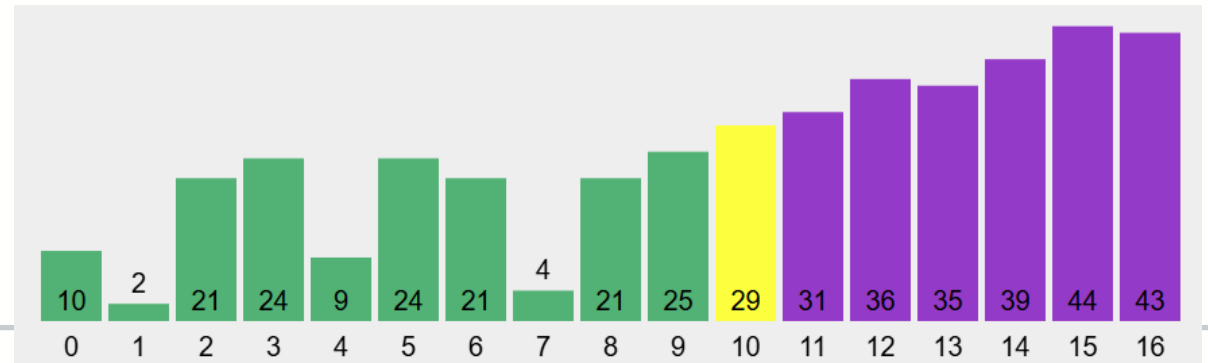
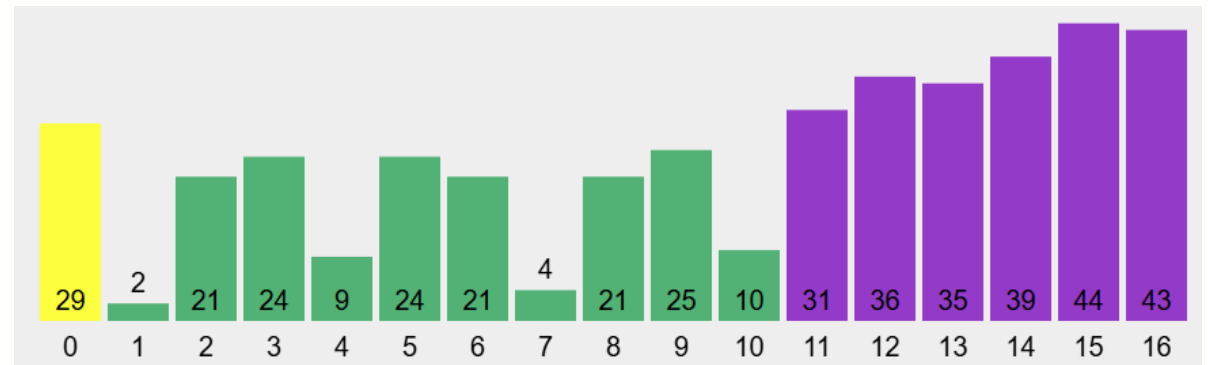
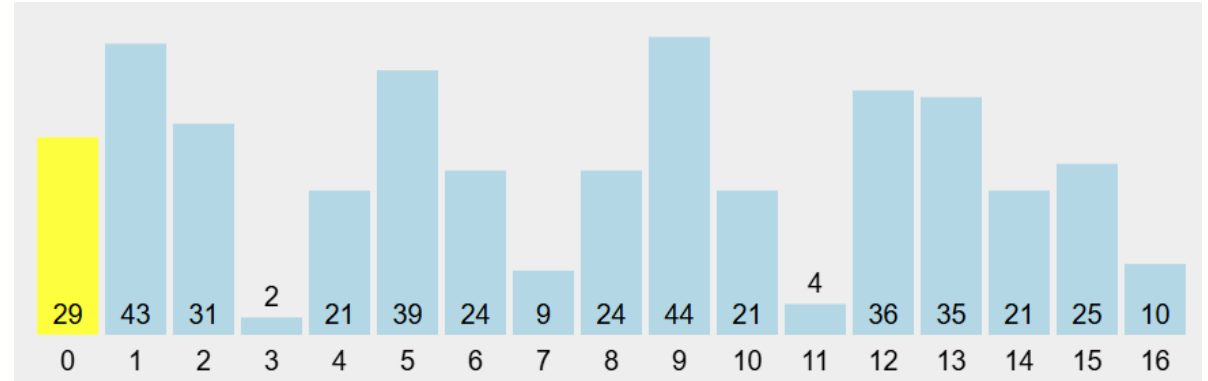
Efficient Selection Algorithms

- Linear time on average (or in the worst case for certain methods)
- Divide-and-conquer algorithm
 - Divide: partition the array into smaller subarrays based on a pivot element
 - Conquer: Recursively solve a smaller subproblem
- Pivot selection methods:
 - Randomized Selection
 - Median of Medians

Review: Quick sort - Partitioning

Quick Sort - Partitioning

- We select a pivot element and arrange the array such that all elements less than the pivot come before the pivot, and all elements greater than the pivot come after the pivot
- You can think of this process as finding the sorted position for the pivot



Partition(A, l, r)

1. $x \leftarrow A[l]$
2. $i \leftarrow l$
3. $j \leftarrow r + 1$
4. while TRUE do
 5. repeat $j \leftarrow j - 1$
 6. until $A[j] \leq x$
 7. repeat $i \leftarrow i + 1$
 8. until $A[i] \geq x$
 9. if $i < j$ then
 10. exchange $A[i] \leftrightarrow A[j]$
 11. else
 12. exchange $A[l] \leftrightarrow A[j]$
 13. return j

Partitioning Example

Partition(A, 1, 8)

1	2	3	4	5	6	7	8
4	3	6	9	2	1	8	5

i

j

1	2	3	4	5	6	7	8
4	3	1	9	2	6	8	5

i

j

1	2	3	4	5	6	7	8
4	3	1	2	9	6	8	5

j

i

1	2	3	4	5	6	7	8
2	3	1	4	9	6	8	5

Quicksort Algorithm Analysis

- General Recurrence Relation

$$T(n) = T(an) + T((1-a)n) + \Theta(n) \Leftarrow \text{partitioning takes } \Theta(n)$$

where a is a fraction (e.g., $a = 0.5$ for balanced splits)

- Base case:

$$T(1) = 1$$

there is only one element in the array (or subarray).

The algorithm returns the array.

Quicksort Analysis for Best Case Partition

- The running time of quicksort depends on whether the partitioning is balanced or not.
- In the best case, each partition operation divides the array in half (exactly)
 - i.e., pivot is the median
- When the partitioning procedure produces two regions of size $\frac{n}{2}$, we get a balanced partition:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

$$\text{So, } T(n) = \Theta(n \log n)$$

- Can it perform better than $O(n \log n)$ on any input?

Quicksort Best Case Partitioning

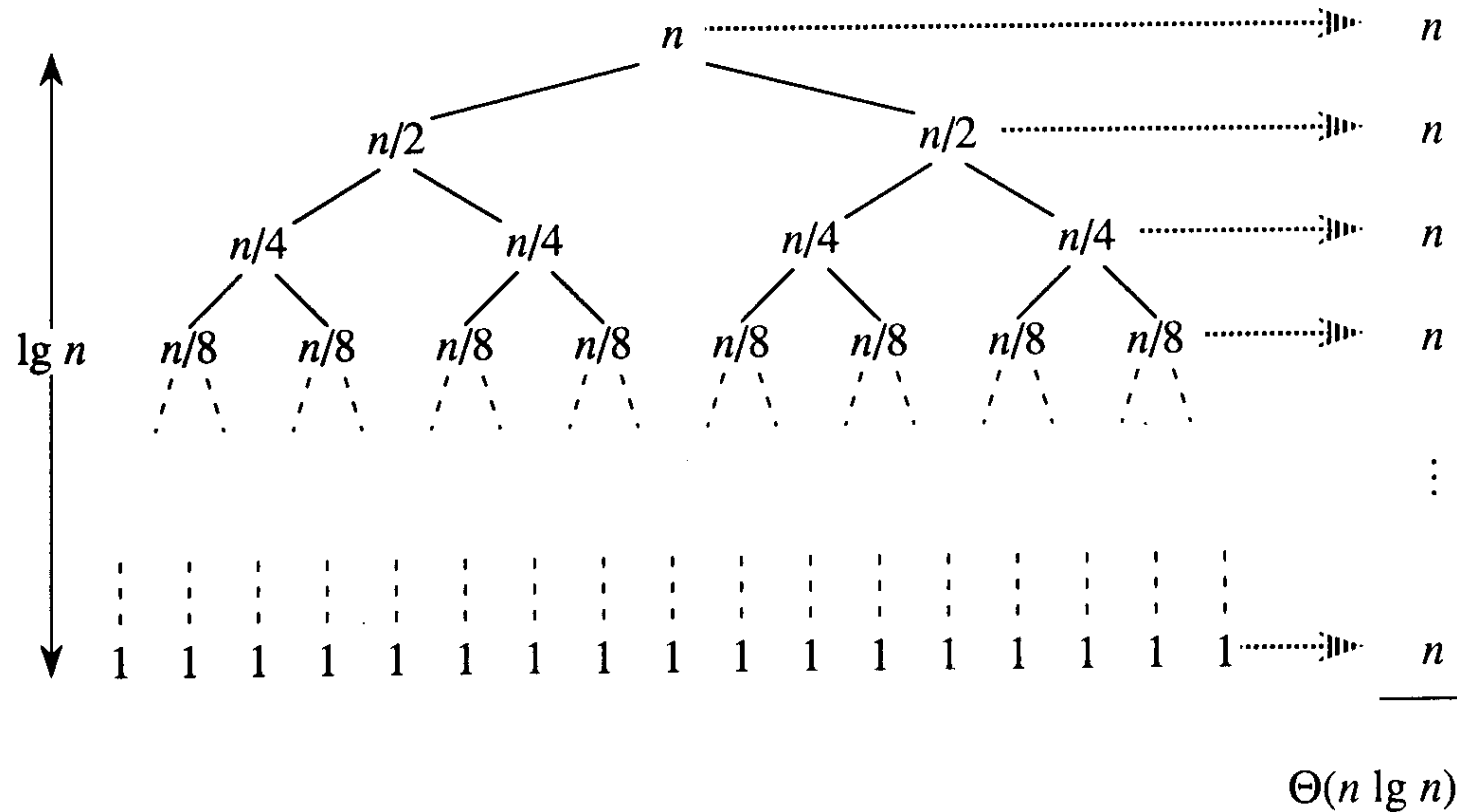


Figure 8.3 A recursion tree for QUICKSORT in which PARTITION always balances the two sides of the partition equally (the best case). The resulting running time is $\Theta(n \lg n)$.

Quicksort Worst Case Partitioning

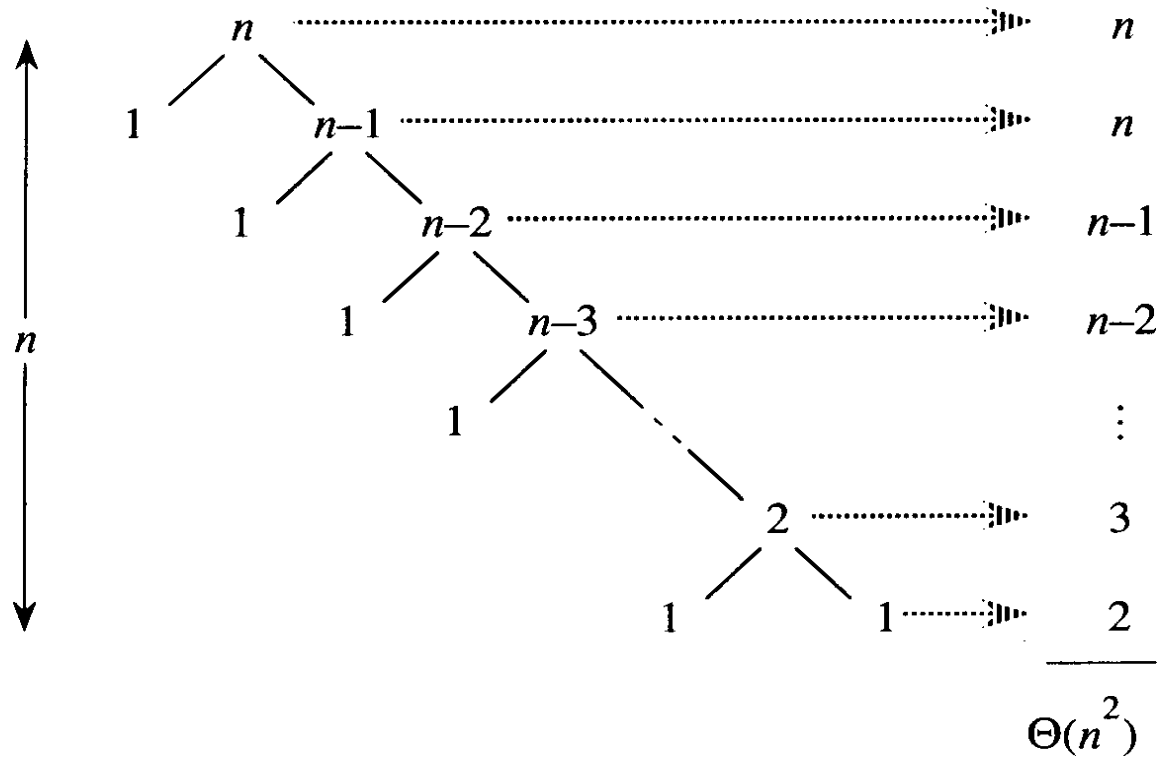


Figure 8.2 A recursion tree for QUICKSORT in which the PARTITION procedure always puts only a single element on one side of the partition (the worst case). The resulting running time is $\Theta(n^2)$.

Randomized-Partition(A, l, r)

1. $i \leftarrow \text{Random}(l, r)$
2. exchange $A[l] \leftrightarrow A[i]$
3. return Partition(A, l, r)

Selection Algorithms

Example – Find the 11th Smallest Element

index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
	a1	a2	a3	a4	a5	a6	a7	a8	a9	a10	a11	a12	a13	a14	a15	a16	a17	a18	a19

16 ← SELECT(A, 1, 19)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
															p			



$\leq A[p]$



$\geq A[p]$

Contains the 1st smallest (minimum) to
15th smallest elements

17th smallest to 19th smallest
(maximum) elements

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

We continue with A[1...15] and ignore A[16...19] because we are looking for the 11th smallest element

Example – Find the 11th Smallest Element

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

4 ← SELECT(A, 1, 15)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
			p															

(a)

(b)

8 ← SELECT(A, 5, 15)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
							p											

Example – Find the 11th Smallest Element

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
							p											

12 ← SELECT(A, 9, 15)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
											p							

10 ← SELECT(A, 9, 11)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
									p									

11 ← SELECT(A, 11, 11)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
										p								

Quickselect

- Works similarly to Quicksort, but instead of fully sorting the array, it focuses only on finding the desired i th smallest element.
- Best case: $O(n)$
- Average case: $O(n)$

$$T(n) = T\left(\frac{n}{2}\right) + O(n)$$

- On average, the pivot partitions the array into two roughly equal halves
- $O(n)$ for partitioning
- Worst case: $O(n^2)$ (if poor pivots are consistently chosen)

Quickselect Average Case

- Average case: $O(n)$

$$\begin{aligned}T(n) &= T\left(\frac{n}{2}\right) + O(n) \\&= O\left(n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots\right) \\&= O\left(n \cdot \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots\right)\right) \\&= O(2n) \\&= O(n)\end{aligned}$$

Discussion

- Quicksort vs. QuickSelect
 - Which is faster?
 - They both use partitioning. What is the main difference (extra work)?
 - Do they have the same worst-case scenario (input sequence)?
- Binary Search vs. QuickSelect
 - Which is faster?
 - After one partitioning step, what do we know about the discarded elements?
 - Do we know their exact order?
- QuickSelect vs. Linear scan
 - When would a simple $O(n)$ scan be a better choice?

Randomized-Select

- Returns the i th smallest element of the array $A[p:r]$, where $1 \leq i \leq r - p + 1$
- Modeled after the randomized-quicksort algorithm
 - Instead of sorting the entire array, it focuses on finding the i th smallest element.
 - Randomized Selection explicitly randomizes the pivot selection, making it more robust on average
- Randomized-Quicksort: Expected running time of $\Theta(n \lg n)$
- Randomized-select:
 - Expected running time: $\Theta(n)$ – averaged over the algorithm's random pivot choices for any input
 - Worst case: $O(n^2)$, but extremely unlikely

Expected time complexity vs. Average-case time complexity

Expected time complexity

Average running time of a randomized algorithm, averaged over all random choices the algorithm makes internally

- Source of randomness: the algorithm (e.g., random pivot)
- Holds for any input – no “bad inputs”, only “unlucky random choices”
- e.g., Randomized Quicksort: even on a sorted list, expected time is $\Theta(n \log n)$ – the random pivot breaks the bad pattern

Average-case time complexity

Average running time of a deterministic algorithm, averaged over all possible inputs of size n

- Source of randomness: the input distribution
- Bad inputs exist – they’re just assumed to be rare
- Example: Deterministic Quicksort: $O(n \log n)$ on average because most input permutations aren’t sorted or verse-sorted

Expected time complexity vs. Average-case time complexity (example)

Expected time

Input: [1,2,3] (fixed)

$$T(n) = (T(\text{pick 1}) + T(\text{pick 2}) + T(\text{pick 3})) / 3$$

Same expected time regardless of input – even the sorted list gives $\theta(n \log n)$

Average-case time

Input: All permutations of {1,2,3}

$$\begin{aligned} T(n) &= (T([1,2,3]) + T([1,3,2]) \\ &\quad + T([2,1,3]) + \dots) / 6 \end{aligned}$$

$O(n \log n)$ on average – but sorted input is still a bad case

Randomized-Select (1)

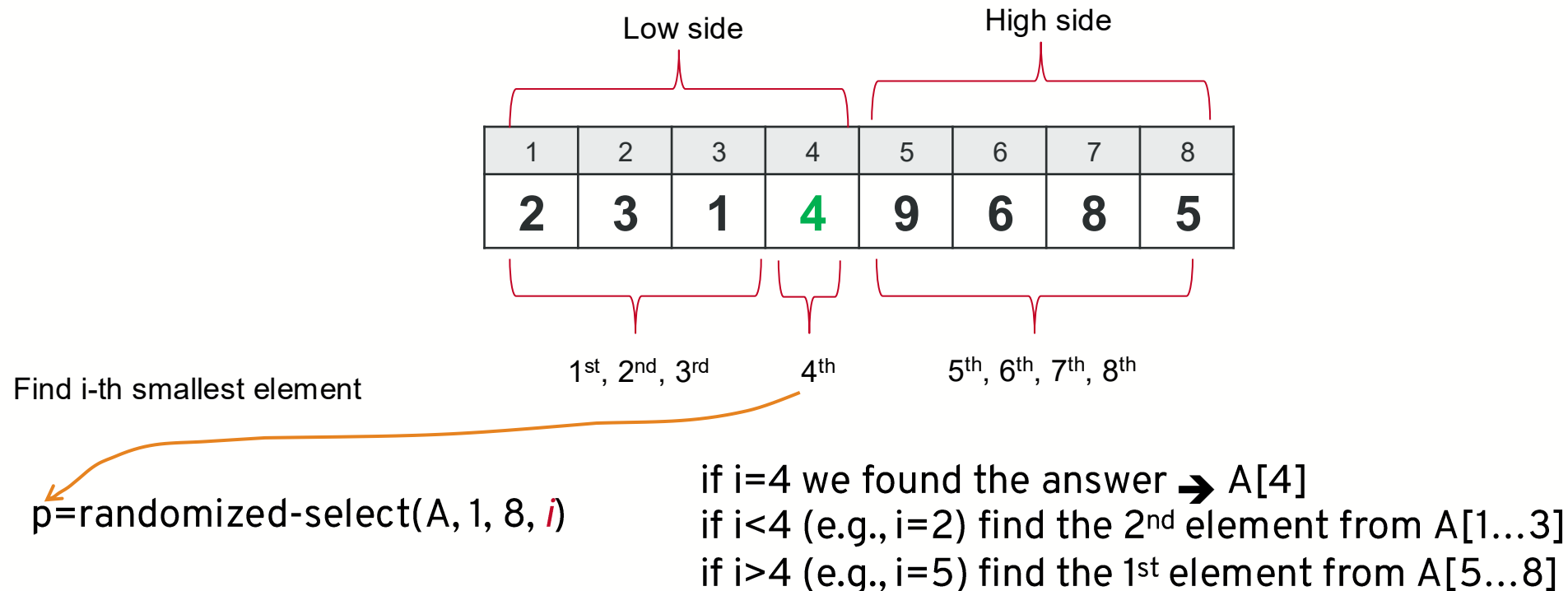
- Partition the input array around a randomly chosen element x using *Randomized-Partition*. Let k be the number of elements on the low side and $n-k$ on the high side.

1	2	3	4	5	6	7	8
2	3	1	4	9	6	8	5

1st, 2nd, 3rd 4th 5th, 6th, 7th, 8th

Randomized-Select

- Use *Randomized-Select* recursively to find the i th smallest element on the low side if $i \leq k$, or the $(i-k)$ th smallest element on the high side if $i > k$



Randomized-Select (A, p, r, i)

```
RANDOMIZED-SELECT( $A, p, r, i$ )
1  if  $p == r$ 
2      return  $A[p]$       //  $1 \leq i \leq r - p + 1$  when  $p == r$  means that  $i = 1$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i == k$ 
6      return  $A[q]$       // the pivot value is the answer
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )
```

Returns the i th smallest element of the array $A[p:r]$, where $1 \leq i \leq r - p + 1$.

The worst-case running time can be $\Theta(n^2)$, but the average performance is $O(n)$.

Randomized-Select Example

A

1	2	3	4	5
8	1	5	3	4

➤ **Goal: Find 3rd smallest element**

Start index $p=1$

End index $r=5$

Find $i=3^{\text{rd}}$ smallest element

`RANDOMIZED-SELECT(A, 1, 5, 3)`

Randomized-Select Example

1	2	3	4	5
8	1	5	<u>3</u>	4

in RANDOMIZED-PARTITION($A, 1, 5$)
 $A[4]$ is randomly chosen as the pivot

RANDOMIZED-SELECT(A, p, r, i)

```
1  if  $p == r$ 
2      return  $A[p]$       //  $1 \leq i \leq r - p + 1$  when  $p == r$  means that  $i = 1$ 
3   $q =$  RANDOMIZED-PARTITION( $A, p, r$ )
4   $k = q - p + 1$ 
5  if  $i == k$ 
6      return  $A[q]$       // the pivot value is the answer
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )
```

Randomized-Partition Example

1	2	3	4	5
8	1	5	<u>3</u>	4

↓

1	<u>3</u>	8	5	4
---	----------	---	---	---

RANDOMIZED-PARTITION(A, 1, 5):

1. Arranges A so that the left side of the pivot is smaller than or equal to the pivot, and the right side of the pivot is greater than or equal to the pivot.
2. Returns the correct index (i.e., sorted position) of the pivot, which is index 2. So, q=2.

RANDOMIZED-SELECT(A, p, r, i)

```
1  if p == r
2      return A[p]    // 1 ≤ i ≤ r - p + 1 when p == r means that i = 1
3  q = RANDOMIZED-PARTITION(A, p, r)
4  k = ...
5  if i = RANDOMIZED-PARTITION(A, p, r)
6      return k
7  elseif i < RANDOMIZED-PARTITION(A, p, r)
8      return RANDOMIZED-SELECT(A, p, q-1, i)
9  else if i > RANDOMIZED-PARTITION(A, p, r)
      return RANDOMIZED-SELECT(A, q+1, r, i)
```

Randomized-Partition Example

1	2	3	4	5
1	<u>3</u>	8	5	4

1st 2nd 3rd, 4th, 5th

$q=2$

$k = q - p + 1 = 2 - 1 + 1 = 2$

We wanted to find the 3rd item ($i=3$)

$i == k$? (False)

$i < k$? (False)

$i > k$? (True)

- We try to find the 1st item in the right subarray:

`RANDOMIZED-SELECT(A, 3, 5, 1)`

`RANDOMIZED-SELECT(A, p, r, i)`

```
1  if p == r
2      return A[p]      // 1 ≤ i ≤ r - p + 1 when p == r means that i = 1
3  q = RANDOMIZED-PARTITION(A, p, r)
4  k = q - p + 1
5  if i == k
6      return A[q]      // the pivot value is the answer
7  elseif i < k
8      return RANDOMIZED-SELECT(A, p, q - 1, i)
9  else return RANDOMIZED-SELECT(A, q + 1, r, i - k)
```

Randomized-Partition Example

1	2	3	4	5
1	3	8	5	4

- We try to find the 1st item in the right subarray:

RANDOMIZED-SELECT(A, 3, 5, 1)

RANDOMIZED-SELECT(A, p, r, i)

```
1  if  $p == r$ 
2      return  $A[p]$       //  $1 \leq i \leq r - p + 1$  when  $p == r$  means that  $i = 1$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i == k$ 
6      return  $A[q]$       // the pivot value is the answer
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )
```

Randomized-Partition Example

1	2	3	4	5
1	3	8	5	<u>4</u>

RANDOMIZED-SELECT(A, 3, 5, 1)
→ RANDOMIZED-PARTITION(A, 3, 5):

RANDOMIZED-SELECT(A, p, r, i)

```
1  if  $p == r$ 
2      return  $A[p]$       //  $1 \leq i \leq r - p + 1$  when  $p == r$  means that  $i = 1$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k$ 
5  if  $\text{RANDOMIZED-PARTITION}(A, p, r)$ 
6
7  1   $i = \text{RANDOM}(p, r)$ 
8  2  exchange  $A[r]$  with  $A[i]$ 
9  3  return  $\text{PARTITION}(A, p, r)$ 
```

Randomized-Partition Example

1	2	3	4	5
1	3	<u>4</u>	5	8

RANDOMIZED-SELECT($A, 3, 5, 1$)

→ RANDOMIZED-PARTITION($A, 3, 5$):

→ returns 3 ($q=3$)

$k=q-p+1 = 3-3+1 = 1$

$i=k$, so we found the 1st smallest item in $A[3...5]$

which is equivalent to the 3rd smallest item in $A[1...5]$

RANDOMIZED-SELECT(A, p, r, i)

```
1  if  $p == r$ 
2      return  $A[p]$       //  $1 \leq i \leq r - p + 1$  when  $p == r$  means that  $i = 1$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i == k$ 
6      return  $A[q]$       // the pivot value is the answer
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )
```

Running Time

- The worst-case running time of Randomized-Select is $\Theta(n^2)$
- The expected running time of Randomized-select is $\Theta(n)$
- No particular input elicits worst case running time.

Selection in Worst-Case Linear Time

- It finds the desired element(s) by recursively partitioning the input array
- Randomized-select has an average-case time complexity of $O(n)$, but its worst-case complexity can degrade to $O(n^2)$ if poor pivots are consistently chosen
- Basic idea: to generate a good split when array is partitioned using a modified deterministic partition
 - Finds the approximate median of an array using the median-of-medians

Median of Medians

- Uses a deterministic method to select a “good” pivot
- It guarantees $O(n)$ time complexity in the worst case because the pivot selection ensures reasonably balanced partitions

Median of Medians (Video)



https://youtu.be/_xp5FdAvIK8?si=yn875Hfwi9GULrl_

Median of Medians

- 1 Divide the n elements of input array into $\lfloor n/5 \rfloor$ groups of 5 elements each and at most one group made up of the remaining $(n \bmod 5)$ elements.
- 2 Find the median of each group by insertion sort & take its middle element (smaller of 2 if even number input).
- 3 Use *Select* recursively to find the median x of the $\lceil n/5 \rceil$ medians found in step 2.
- 4 Partition the input array around the median-of-medians x using a modified *Partition*. Let k be the number of elements on the low side and $n-k$ on the high side.
- 5 Use *Select* recursively to find the i th smallest element on the low side if $i \leq k$, or the $(i-k)$ th smallest element on the high side if $i > k$

SELECT(A, p, r, i)

```
1  while  $(r - p + 1) \bmod 5 \neq 0$ 
2      for  $j = p + 1$  to  $r$                 // put the minimum into  $A[p]$ 
3          if  $A[p] > A[j]$ 
4              exchange  $A[p]$  with  $A[j]$ 
5      // If we want the minimum of  $A[p : r]$ , we're done.
6      if  $i == 1$ 
7          return  $A[p]$ 
8      // Otherwise, we want the  $(i - 1)$ st element of  $A[p + 1 : r]$ .
9       $p = p + 1$ 
10      $i = i - 1$ 
11      $g = (r - p + 1) / 5$                 // number of 5-element groups
12     for  $j = p$  to  $p + g - 1$             // sort each group
13         sort  $\langle A[j], A[j + g], A[j + 2g], A[j + 3g], A[j + 4g] \rangle$  in place
14     // All group medians now lie in the middle fifth of  $A[p : r]$ .
15     // Find the pivot  $x$  recursively as the median of the group medians.
16      $x = \text{SELECT}(A, p + 2g, p + 3g - 1, \lceil g/2 \rceil)$ 
17      $q = \text{PARTITION-AROUND}(A, p, r, x)$  // partition around the pivot
18     // The rest is just like lines 3–9 of RANDOMIZED-SELECT.
19      $k = q - p + 1$ 
20     if  $i == k$ 
21         return  $A[q]$                     // the pivot value is the answer
22     elseif  $i < k$ 
23         return SELECT( $A, p, q - 1, i$ )
24     else return SELECT( $A, q + 1, r, i - k$ )
```

Lines 1-10: sort the last group if they have fewer than 5 elements

Lines 11-13: sort each group

Line 16: find median of medians recursively. This is our special pivot (x).

Line 17:

Partition: Rearrange all items around x , so smaller items are on the left and larger ones are on the right. This helps us zero in on the k -th smallest item faster.

Line 19-24:

Recalculate k : Count the items on the left of x .

Recurse: Repeat the process on the side (left or right of x) that contains the k -th smallest item we're after.



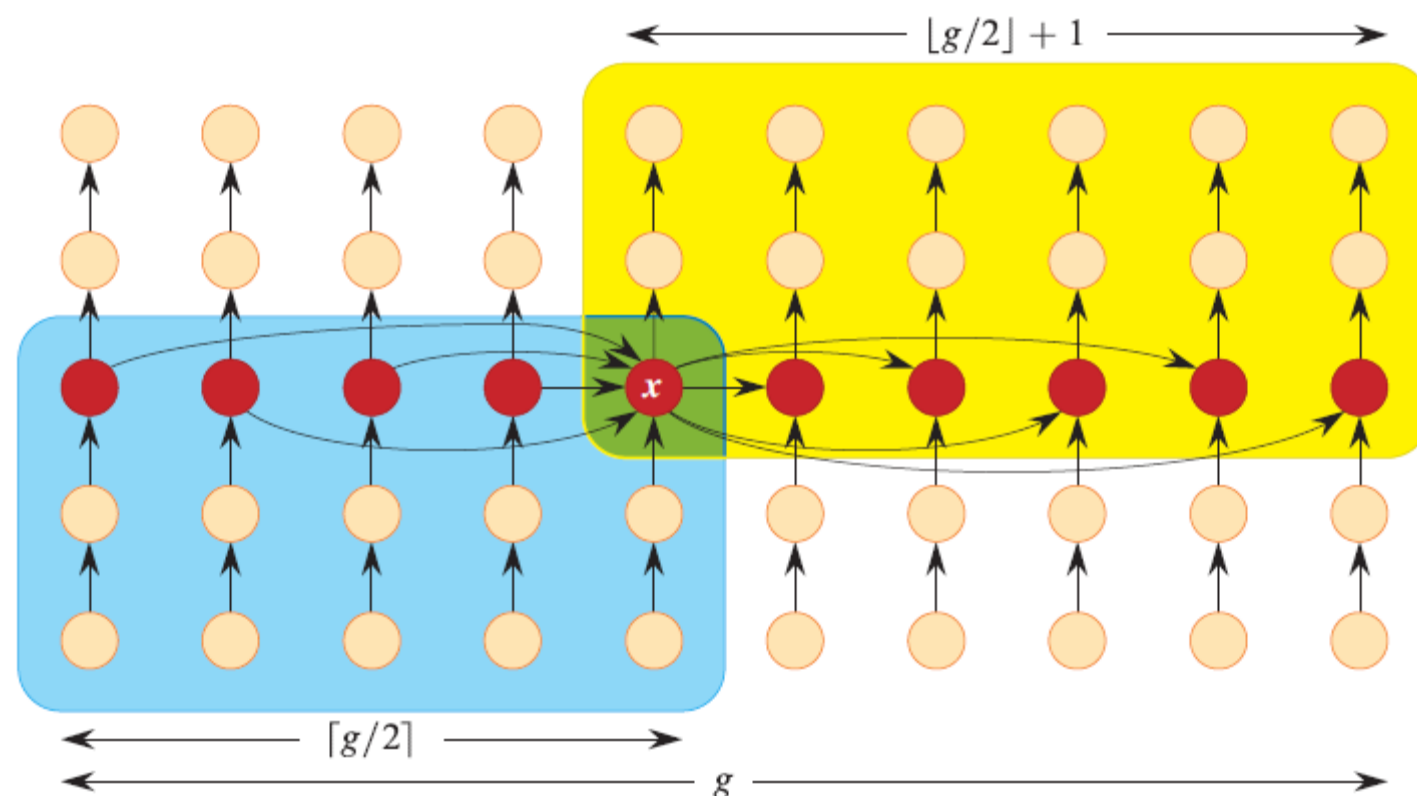


Figure 9.3 The relationships between elements (shown as circles) immediately after line 17 of the selection algorithm SELECT. There are $g = (r - p + 1)/5$ groups of 5 elements, each of which occupies a column. For example, the leftmost column contains elements $A[p]$, $A[p + g]$, $A[p + 2g]$, $A[p + 3g]$, $A[p + 4g]$, and the next column contains $A[p + 1]$, $A[p + g + 1]$, $A[p + 2g + 1]$, $A[p + 3g + 1]$, $A[p + 4g + 1]$. The medians of the groups are red, and the pivot x is labeled. Arrows go from smaller elements to larger. The elements on the blue background are all known to be less than or equal to x and cannot fall into the high side of the partition around x . The elements on the yellow background are known to be greater than or equal to x and cannot fall into the low side of the partition around x . The pivot x belongs to both the blue and yellow regions and is shown on a green background. The elements on the white background could lie on either side of the partition.

Algorithm Analysis (1)

- At least half of the medians found in step 2 are greater or equal to the median-of-medians x (by definition)
- Elements greater than x : Each group of five has a median, and if the median is greater than x , then at least three elements in that group are greater than x (the median itself and two elements greater than the median)
 - Half of the $n/5$ groups have medians greater than or equal to x . So, the number of groups with medians greater than x is at least $n/10$
 - Each of these groups contributes at least 3 elements greater than x , except for the group containing x itself (which contributes only 2 elements greater than x), and the group with fewer than 5 elements
 - We will exclude the group with fewer than 5 elements, and the group containing the pivot x itself from this analysis
- Thus, the number of elements greater than x is at least:

$$3 \left(\left\lceil \left(\frac{1}{2} \right) \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right) \geq 3 \left(\left\lceil \frac{n}{10} \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$$

Algorithm Analysis (2)

- Similarly, the number of elements $< x$ is at least $\frac{3n}{10} - 6$
- In the worst case, the SELECT algorithm is called recursively on the larger portion of the array, at most $\frac{7n}{10} + 6$
- Total elements = n
- Worst-case remaining elements to consider: $n - (\frac{3n}{10} - 6) = \frac{7n}{10} + 6$
- The algorithm reduces the problem size by a constant fraction $0.7n$ in each recursive call, leading to a linear number of comparisons in total

$$n + 0.7n + 0.7^2n + \dots = n \sum_{i=0}^{\infty} 0.7^i = \frac{n}{1 - 0.7} = \frac{n}{0.3} = \frac{10n}{3}$$

Algorithm Analysis (3)

- Step 1: Initial grouping and any necessary adjustments $O(n)$
- Step 2: Sorting groups and finding medians, $O(1) \cdot \left\lceil \frac{n}{5} \right\rceil = O(n)$
- Step 3: Finding the median of medians takes time $T\left(\left\lceil \frac{n}{5} \right\rceil\right)$
- Step 4: Partitioning the array around the pivot $O(n)$
- Step 5: Recursive Call $T\left(\frac{7n}{10} + 6\right)$
- Finding the median of the medians by recursively applying the SELECT algorithm.
 - $T(n) \leq \Theta(1)$, if $n \leq 80$ (base case)
 - $T(n) \leq T\left(\left\lceil \frac{n}{5} \right\rceil\right) + T\left(\frac{7n}{10} + 6\right) + O(n)$, if $n > 80$
 - If $n \leq 80$, the algorithm may select the i th smallest element using a simpler method, such as sorting the entire array and then selecting the i th element.
 - 80 is a practical threshold based on empirical observations, where the overhead of the median-of-medians algorithm starts to outweigh its benefits for smaller arrays

Solving Recurrence

- Substitution Method: Guess $T(n)=O(n)$, Prove $T(n) \leq cn$

$$\begin{aligned} T(n) &\leq c \left\lceil \frac{n}{5} \right\rceil + c \left(\frac{7n}{10} + 6 \right) + O(n) \\ &\leq c \left(\frac{n}{5} + 1 \right) + c \left(\frac{7n}{10} + 6 \right) + O(n) \\ &= \frac{cn}{5} + c + \frac{7cn}{10} + 6c + O(n) \\ &\leq \frac{9cn}{10} + 7c + O(n) = cn - \left(c \left(\frac{n}{10} - 7 \right) - O(n) \right) \\ &\leq cn \iff \text{if we choose } c \text{ large enough such} \\ &\quad \text{that } c \left(\frac{n}{10} - 7 \right) \text{ is larger than } O(n), n > 80 \end{aligned}$$

Summary

	Randomized Selection	Median of Medians
Time Complexity	Average Case: $O(n)$ Worst Case: $O(n^2)$	Average Case: $O(n)$ Worst Case: $O(n)$
Pivot Selection	Random pivot selection	Deterministic pivot selection Ensures pivot is within 30th to 70th percentile
Use Cases	Large datasets where average-case performance is key	When worst-case performance is critical
Efficiency	Generally faster in practice due to simplicity	Slower in practice due to additional computations
Advantages	Simplicity Efficiency on average Low overhead	Guaranteed worst-case performance No bias in pivot selection
Disadvantages	No worst-case guarantee Potential for clustering	Higher overhead due to grouping, sorting, and recursion More complex to implement Slower in practice